Estructuras de Datos

2026

Primer Cuatrimestre

Motivaciones

El árbol binario de búsqueda permite implementar conjuntos y mapeos con un tiempo de operaciones buscar, insertar y eliminar con orden logarítmico en la cantidad de elementos en promedio.

En el peor caso las operaciones tienen orden lineal en la cantidad de elementos (cuando las inserciones se realizaron en forma ascendente o descendente en cuyo caso el árbol degenera en una lista).

Hay estructuras alternativas que garantizan tiempo de acceso de orden logarítmico en la cantidad de elementos y se los conoce como árboles de búsqueda balanceados: Árbol AVL, Árbol 2-3 y Árbol B, entre otros.

Árboles AVL

Agregaremos una corrección al árbol binario de búsqueda para mantener una altura del árbol proporcional al logaritmo de la cantidad de nodos del árbol.

Recordemos que el tiempo de búsqueda, inserción y borrado en un árbol binario de búsqueda es lineal en la altura del árbol.

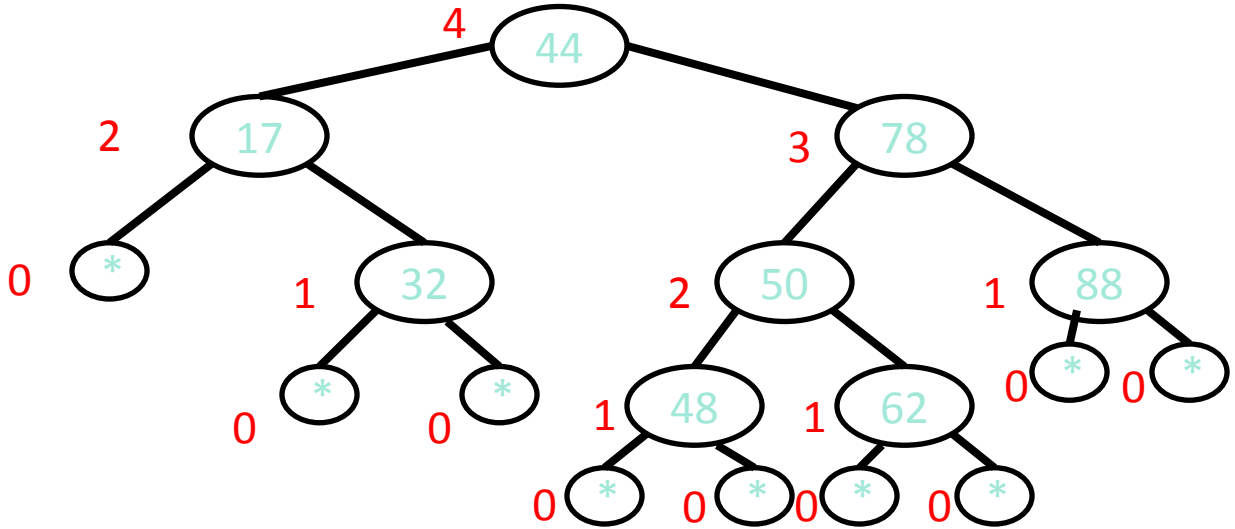
Entonces, si n=cantidad de elementos de un árbol T, tendríamos así que $T(n) = O(\log_2(n))$.

Propiedad del balance de la altura: Para cada nodo interno v de T, las alturas de los hijos difieren en a lo sumo 1.

Cualquier árbol binario de búsqueda que satisface esta propiedad se dice "árbol AVL" (por Adel'son-Vel'skii y Landis).

Árboles AVL: Ejemplo

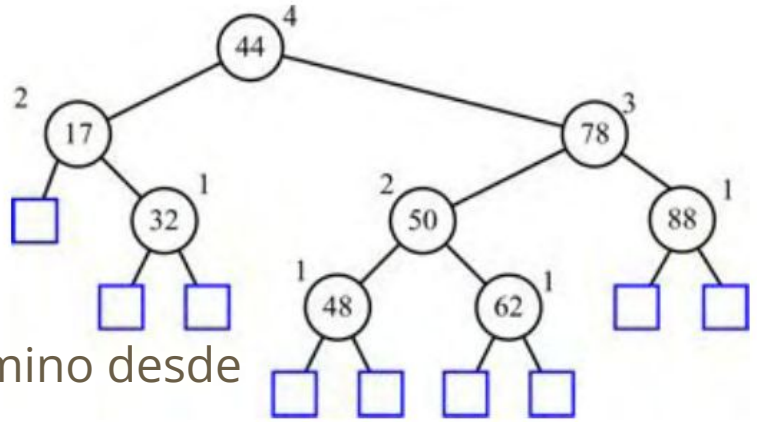
- Propiedad del balance de la altura: Para cada nodo interno v de T, las alturas de los hijos difieren en a lo sumo 1.
- Ejemplo: Los * corresponden a nodos nulos (con altura 0 de acuerdo a GT).



Comentarios

- Siguiendo a Goodrich & Tamassia diremos que los nodos nulos tienen altura 0.
- Como un AVL es un árbol binario de búsqueda, la operación de búsqueda no sufre alteraciones.
- La únicas operaciones a modificar son la de inserción y eliminación, las cuales deben verificar que se cumpla la propiedad de balance al finalizar la operación.
- Luego de cada modificación en un nodo, rebalancearán los hijos de dicho nodo en O(1) por medio de las llamadas “rotaciones”.
- Las modificaciones se hacen desde la hoja donde se insertó el nodo hacia la raíz siguiendo el camino de llamadas recursivas.
- Las eliminaciones las haremos perezosas (lazy): marcaremos los datos eliminados con un booleano.

Rotación



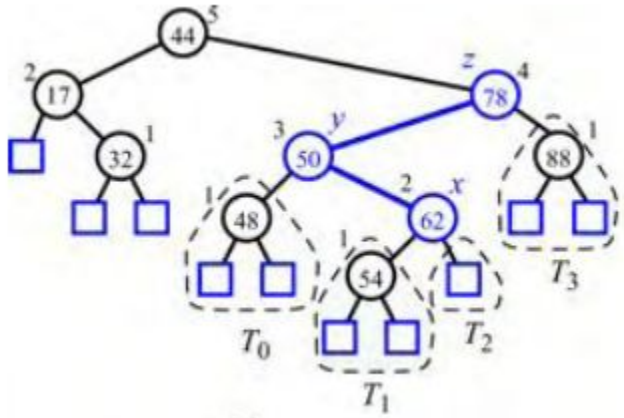
Supongamos que agrego el 54

Solo la altura de los nodos que están en el camino desde la raíz hasta el nodo agregado cambian

z: primer nodo yendo hacia la raíz que está desbalanceado

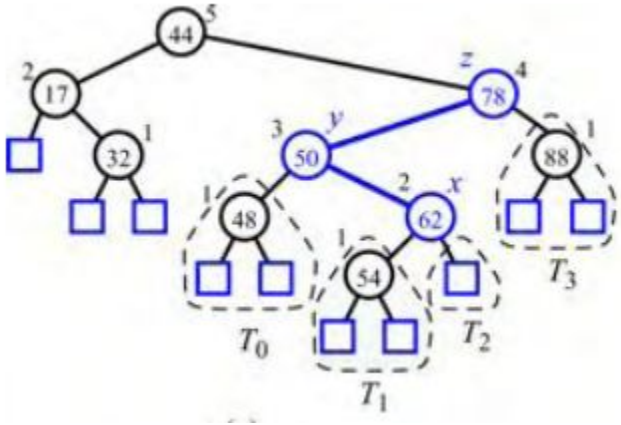
y: hijo de z con la mayor altura

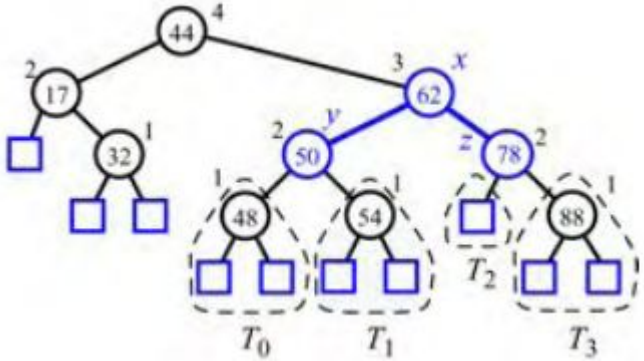
x: hijo de y con la mayor altura



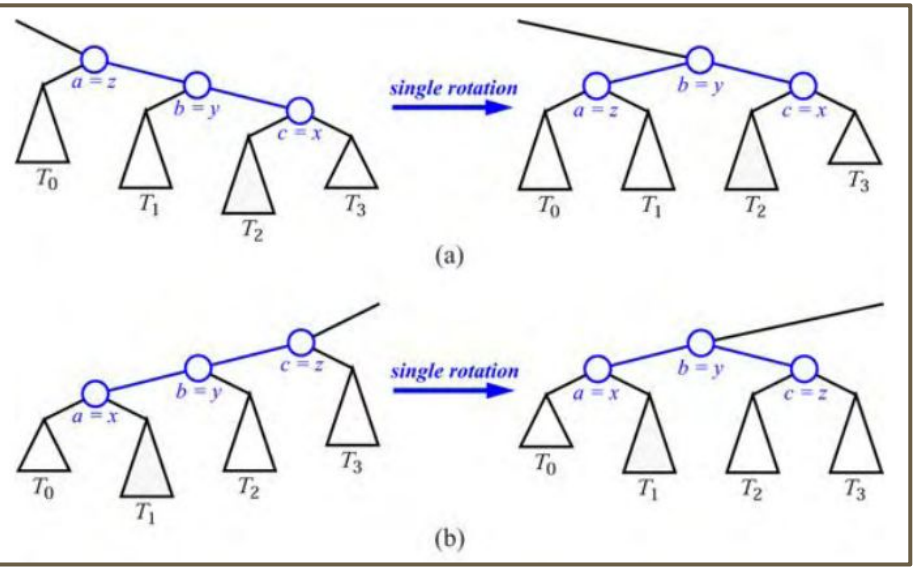
z se volvió desbalanceado por una inserción en el hijo y. La altura de y es 2 más grande que la de su hermano

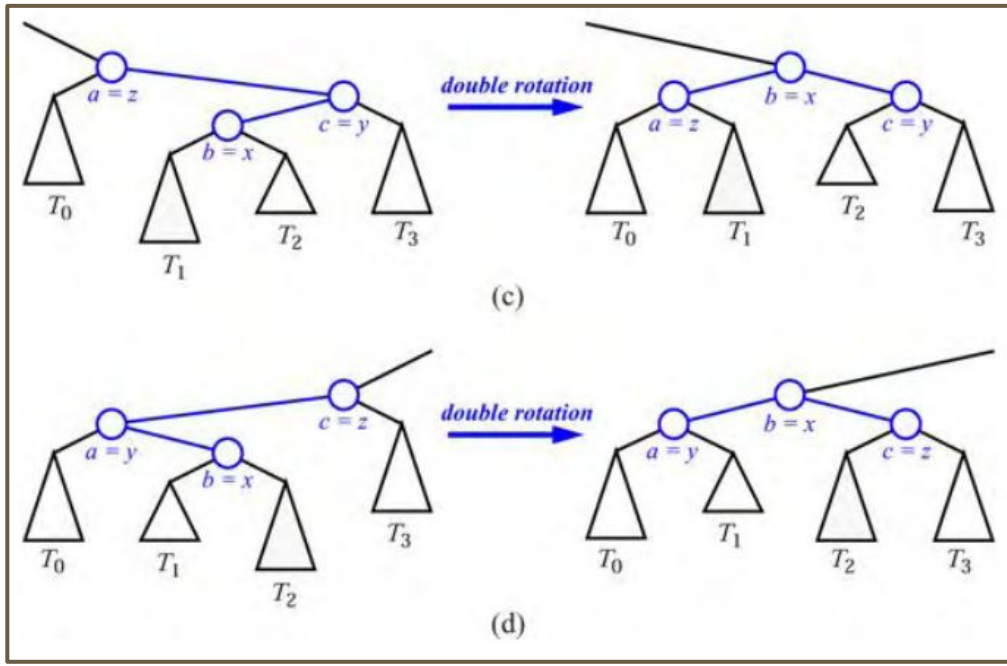
Rotación


44
17
32
78
50
88
48
62
54
z
y
x
T_0
T_1
T_2
T_3


44
17
32
62
50
78
48
54
88
x
y
z
T_0
T_1
T_2
T_3

Posibilidades





Complejidad temporal de la inserción

Noten las rotaciones se hacen en los nodos del camino desde la raíz hasta la hoja donde se insertó la nueva clave.

Como las rotaciones se implementan con asignaciones de referencias (posiciones), cada rotación se hace en tiempo constante.

La altura es proporcional al logaritmo de la cantidad de nodos del árbol.

Por lo tanto, el tiempo de insertar es del orden del logaritmo de la cantidad de nodos del árbol.